Paws & Claws Design Documentation

Team Information

- Team name: HalfCourt
- Team members
  - Jonah Witte
  - Kayla Van Bortel
  - Max Klot
  - Ryan Richter

Executive Summary

The New York State Paws & Claws U-Fund is an application that encourages Helpers to donate money towards supporting animals. They do this by checking out Needs, such as a donation that goes towards a bag of dog food, and fund them through their basket. Helpers are incentivized by a leveling and badge-earning system and can connect with each other by filling out their profile and viewing others'. Managers can add, edit, and delete the Needs from the cupboard, and are also able to view statistics from Helpers' profiles to see which audiences to market more towards.

Purpose

The purpose of our U-Fund application is to allow philanthropists to be able to fund the NYS Paws & Claws foundation, and for administrators of the NYS Paws & Claws Foundation to be able to manage the application.

Glossary and Acronyms

Term	Definition
SPA	Single Page
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
DAO	Data Access Object
ID	Identifier
UUID	Universally Unique Identifier
REST	Representational State Transfer
OO	Object-Oriented
rank	A helper's position on the leaderboard (rank 1 is best)
level	Status based on donations made (higher level = better)

Requirements

This section describes the features of the application.

Definition of MVP

The user is first routed to a login page. If they enter an new username, an account is created for them and they are sent to the homepage. If they enter a username that already exists, they are logged in and sent to the homepage. If they enter "admin", they are sent a Manager version of the homepage. On the homepage, you can see a list of needs in the cupboard.Listed below is the functionality available to Helpers and Managers:

Helper

-you are able to search for needs -add needs to your basket, and checkout.

Manager

-you can add, modify, and delete needs -you do not have a personal funding basket

Data persists from all actions of the Helper and Manager across the application.

MVP Features

- Authentication
  - Helper login
  - Manager login
- Web Application
  - Visit webpage
  - Search needs
- Helper Donations
  - Add donation needs to basket
  - View basket needs
  - Remove donation needs from basket
  - Check out donation needs
- Donation Management
  - Add new needs
  - Update needs
  - Delete needs

Enhancements

Gamification

- In order to encourage Helpers to donate, we gamified the U-Fund application.
- A leaderboard tracks Helpers' donations and a Helper is encouraged to donate more to move up the ranks and beat their competition. The top three Helpers on the leaderboard are displayed on the front page for all to see.
- At the top of the leaderboard is the U-Fund God, who is the most recent Helper to have donated the last available item in the Need cupboard, emptying it. This coveted title gets competitive when Needs start to dwindle.

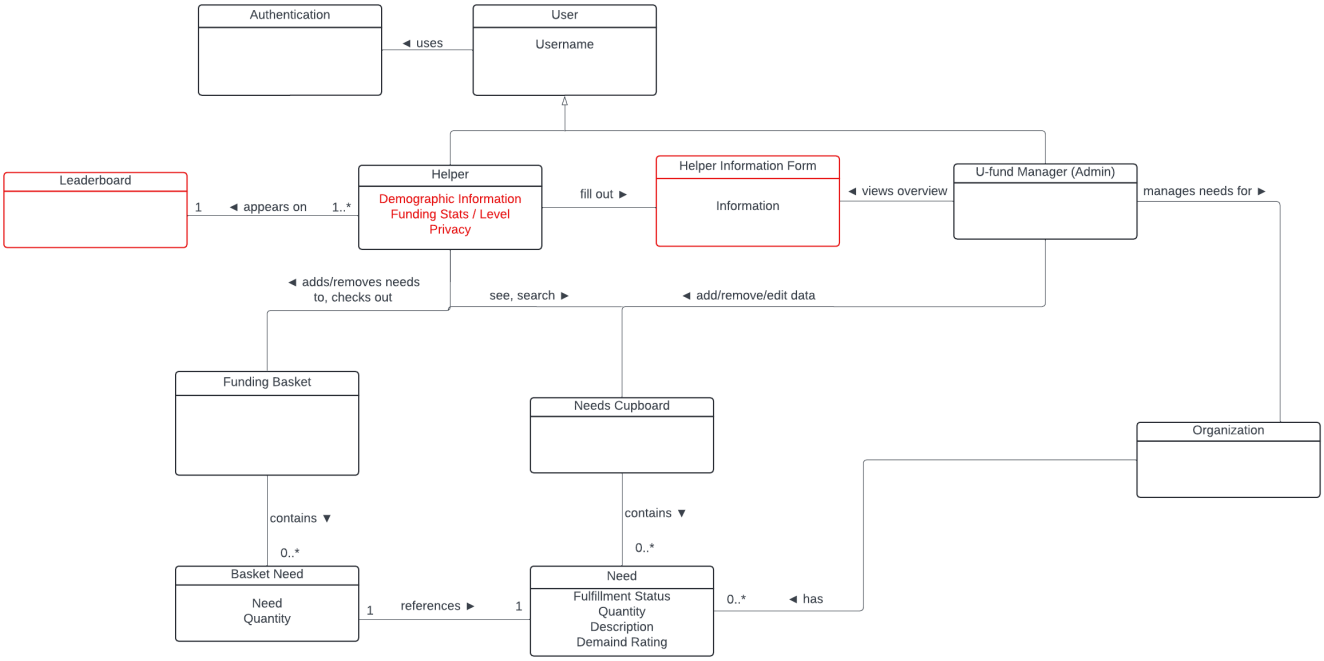
- A Helper's level and donation statistics are displayed on their profile page with a progress bar showing the criteria for them to reach the next level and earn their next collectible badge. Levels include Noob, Pro, Master, and Champion.

Profile Info

- A Helper can update their profile information and set their profile to public so that others can view them and learn more about them. A Helper can view others' profile info from the leaderboard, with a chance to connect with them if that Helper has provided their email or phone number.
- Using the profile info provided by Helpers, a Manager can view Helper statistics with visualized breakdowns on the Admin Dashboard.

Application Domain

This section describes the application domain.



The domain for this application is a New York State based wildlife and animal conservation/refuge organization called NYS Paws & Claws. The application will have:

- Users, including:
    - Helpers
    - Manager
- Needs
- Cupboard
- Baskets
- Helper leaderboard
- Admin dashboard

MVP

A Helper or Manager uses authentication to log in.

A Helper has a basket.

A Need goes in the cupboard and a basket.

A Helper searches for needs in the cupboard.

A Helper adds, removes, and checks out needs from their basket.

An Admin adds, edits, and deletes needs from the cupboard.

Enhancements

A Helper appears on the leaderboard (with a profile picture).

A Helper fills out the profile information form on their profile page.

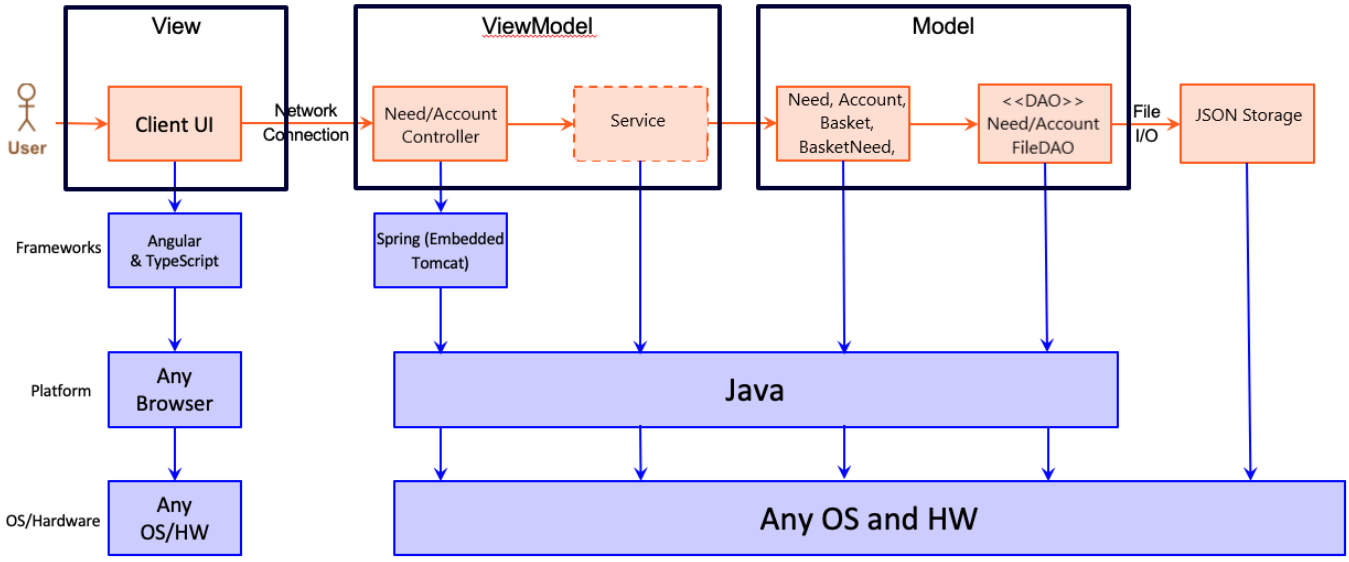
A Manager sees the admin dashboard with a summary of important info from the profile information forms.

Architecture and Design

This section describes the application architecture.

Summary

The following Tiers/Layers model shows a high-level view of the webapp's architecture. **NOTE**: detailed diagrams are required in later sections of this document.



The web application is built using the Model–View–ViewModel (MVVM) architecture pattern.

The Model stores the application data objects including any functionality to provide persistance.

The View is the client-side SPA built with Angular utilizing HTML, CSS and TypeScript. The ViewModel provides RESTful APIs to the client (View) as well as any logic required to manipulate the data objects from the Model.

Both the ViewModel and Model are built using Java and Spring Framework. Details of the components within these tiers are supplied below.

Overview of User Interface

This section describes the web interface flow; this is how the user views and interacts with the web application.

On page load, a user is met with a login page, prompting them to enter a username. After entering a username, their username is saved to the application (if not admin), and a new account is created for the user. Then, a user is redirected to the cupboard page, where they can search, select, and add needs to their basket. They can also access their basket with a button, where they can edit quantities of needs within their basket. If a user logs in as admin, they are able to edit the needs cupboard and cannot view the funding basket. If at any time a user refreshes their page, their authentication is lost and they will have to sign in again.

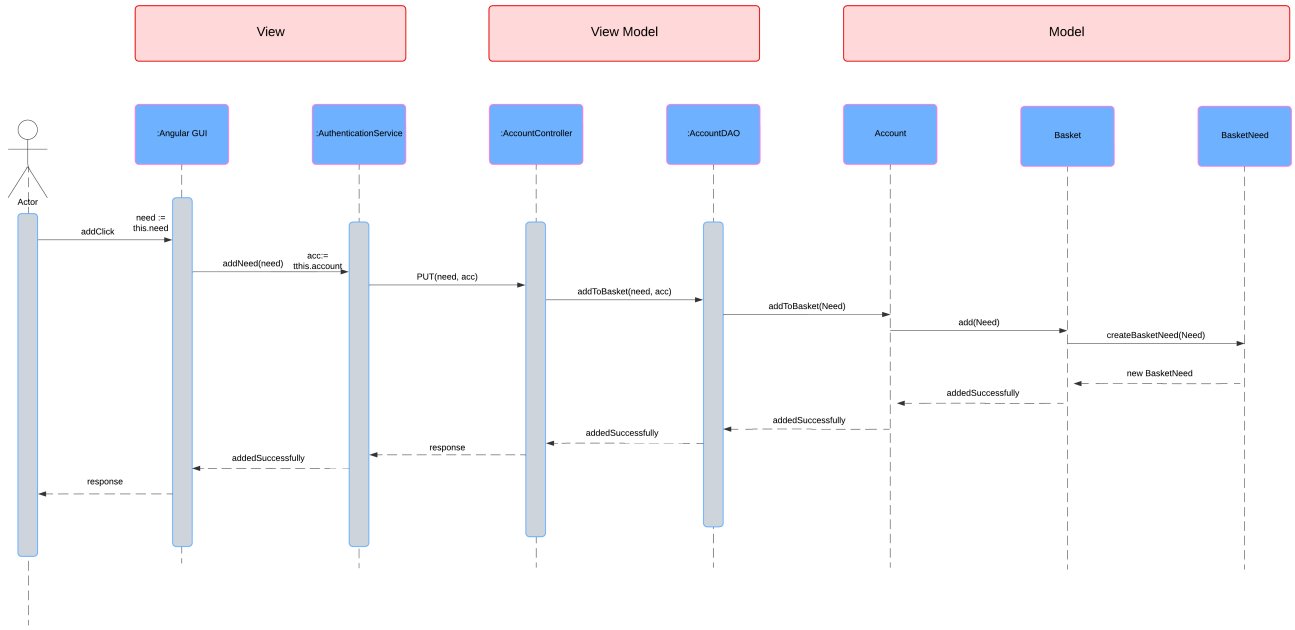
View Tier

The user is first presented with the login page. Here, they enter their username and password in input boxes and click the "Login" button.

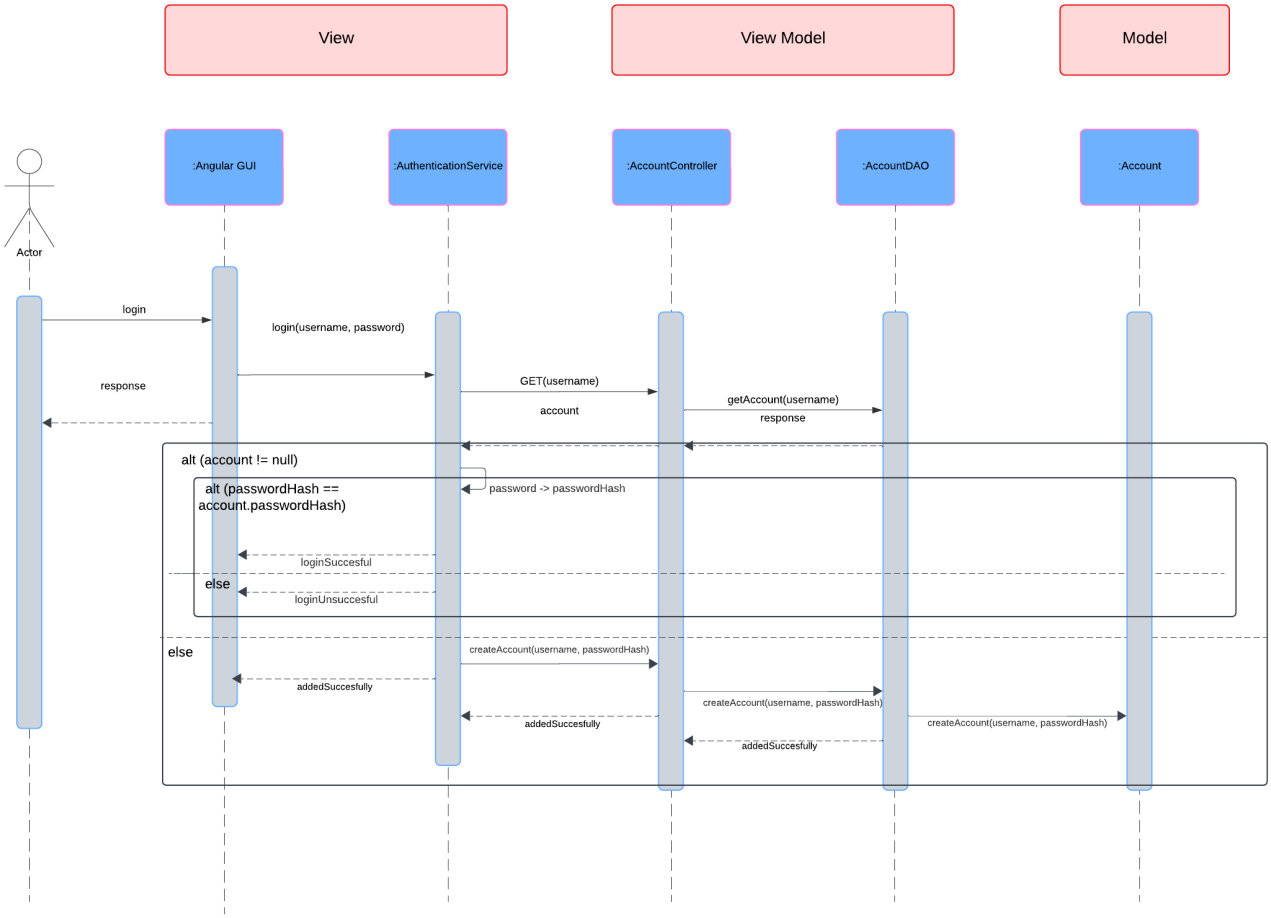
A Helper logs in.
On the home page, they are greeted with a leaderboard on the left displaying the top 3 donors and the U-Fund God. On the right, they see a list of needs with a search bar above to find specific ones. Each need has an add-to-basket button, which, when clicked, displays a notification on that right that tells the Helper the quantity of that need now in their basket.
If the Helper clicks on their "View Basket" button, they see the list of needs and their quantities in their basket. The quantities can be edited, or they can click the trash can button to delete the need from their basket with a prompt asking if they are sure. Then they can click "Fund It!" and the needs will be funded. They click the button to return home.
Back on the home page, they can click on names on the leaderboard to see that Helper's profile information. They click the home button to return to the homepage. The Helper can also click on their profile picture to logout, view the expanded leaderboard, or view their profile.
In their profile, a Helper can upload their profile image with the "Upload Photo" button, click "Edit Profile" and fill out the form fields to add information to the profile with "Save" and "Cancel" buttons, toggle their profile between private and public, or click "Show Stats" to see their donation and level statistics.

The Helper logs out. A Manager logs in with username "admin" and password "adm1n!".
With no profile or basket for the Manager, they only see a logout button in the top right corner.
On the left side of the homepage, there is the searchable list of needs, but now each need has a delete and edit button, prompting a confirmation dialog or a fillable form, respectively.
Beside the search bar is an "Add Need" button, which prompts a fillable form to add a new need.
On the right side of the homepage, a Manager sees Helper statistics, including the total Helpers, needs funded, and amount funded, as well as a pie chart breakdown for Helper regions, funding by region, and the leaderboard, which are navigated to with different tabs in the dashboard.

Sequence Diagram #1: Helper adding a need to their funding basket



Sequence Diagram #2: Helper logging in



ViewModel Tier

The main classes for our ViewModel implementation are our NeedController and AccountController classes. These classes serves to interact with the our DAO interfaces.

AccountController
When a Helper enters a new username and password, a request to "createAccount" is made. This uses an AccountRequest in the parameter to add a new account with the unique name.
When a Helper adds or subtracts from the quantity of a need in their basket, "updateNeed" is called, which updates the quantity of their need in the basket.
When a Helper checks out their basket, "checkout" is called, which uses the accountDAO to check out the needs.
When a Helper views their basket, "getNeeds" retrieves all the needs associated with their account.
When a Helper views their profile, "getProfileInfo" is called to display the information they have previously entered.
When a Helper updates their profile information, "updateProfileInfo" is called to make these changes associated with their account in the ProfileInfo object.
When a Helper uploads a new image to be their profile picture, "handleImageUpload" is called with the account name and file.
"getAccount" retrieves a specific account, "getAccountsSorted" retrieves the accounts in a sorted order for the leaderboard, "getRank" retrieves a Helper's leaderboard rank, "getGod" retrieves whether they are of U-Fund God status, and "getAdminInfo" retrieves the collection of Helper statistics to display on the Manager dashboard.

NeedController
When a Manager adds a new need, "createNeed" is called to add this need to the DAO.
When a Manager edits a need, "updateNeed" is called to persist this new information.
When a Manager deletes a need, "deleteNeed" is called to permanently remove this need from storage.
When a Helper or Manager uses the search bar to search for a need, "searchNeeds" is called to retrieve the given needs where the text matches.
"getNeed" retrieves a single need by id, and "getNeeds" retrieves all needs to be displayed in the cupboard.

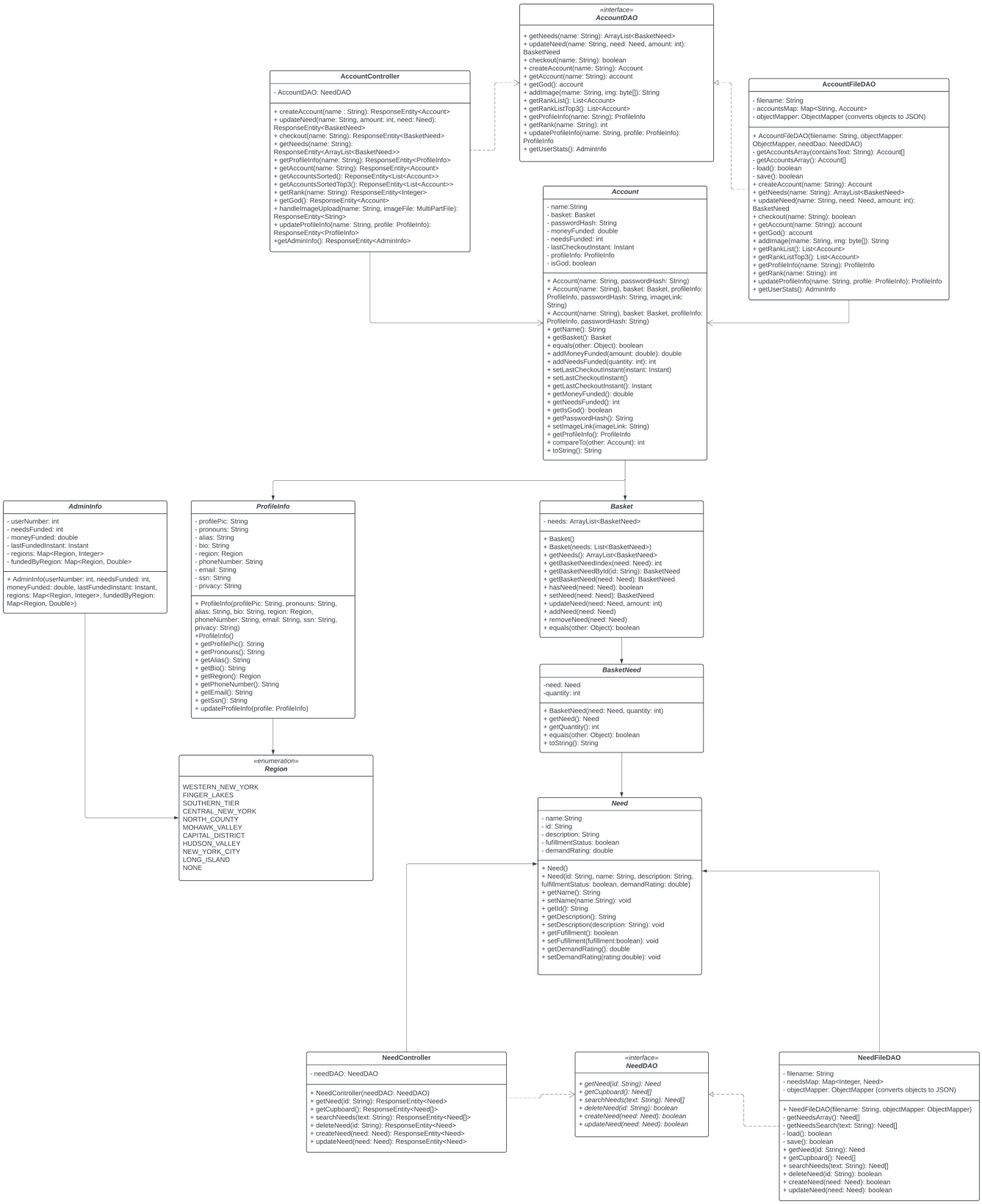
Model Tier

The basic data class for our Model Tier implementation is our Need class. It has generalized members and functions including name, id, description, demand rating, cost, quantity, and fullfillment status.

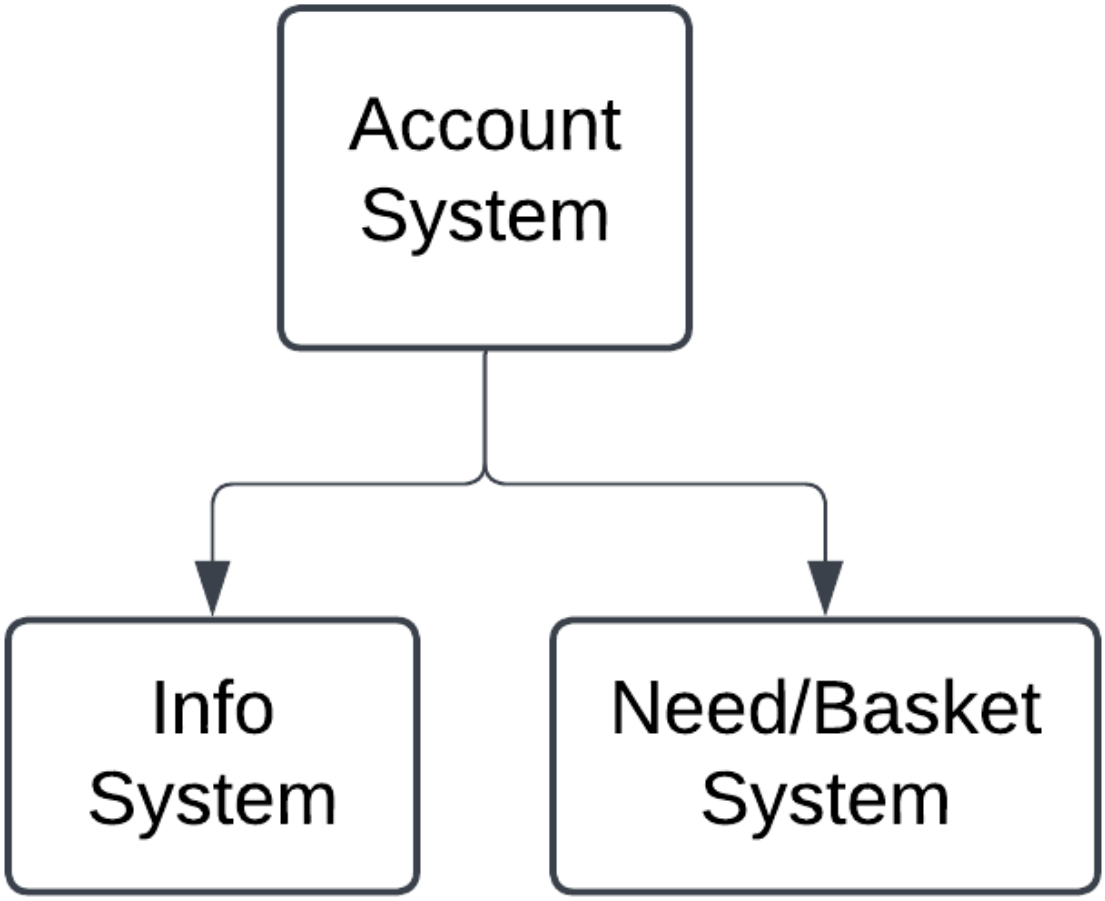
In Sprint 2, we decided to implement three new classes within our model tier, the Account and Basket and BasketNeed classes. The Account class represents an account, which has a name and a basket of needs. The basket of needs contains an arraylist of basketneeds, which are object representations of needs that stay inside a user's basket.

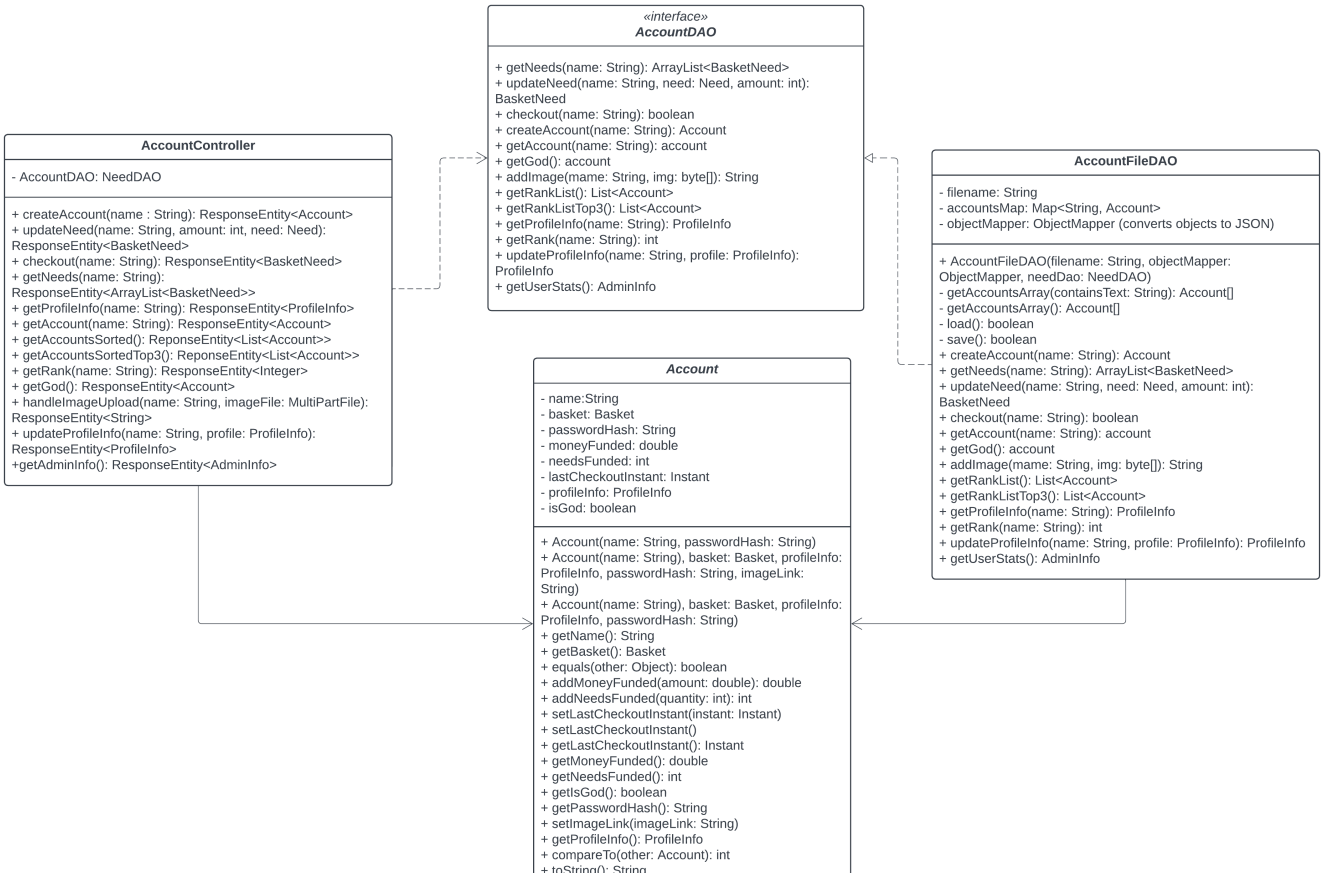
By the end of the project, our Model tier includes Account, AdminInfo, Basket, BasketNeed, Need, ProfileInfo, and Region. AdminInfo groups Helper statistics in a way that they can be usefully displayed to the Manager in their dashboard. Similarly, ProfileInfo groups the information that a Helper enters into their profile to be displayed for themselves and for other users to see if they desire. Region is an enum used by ProfileInfo.

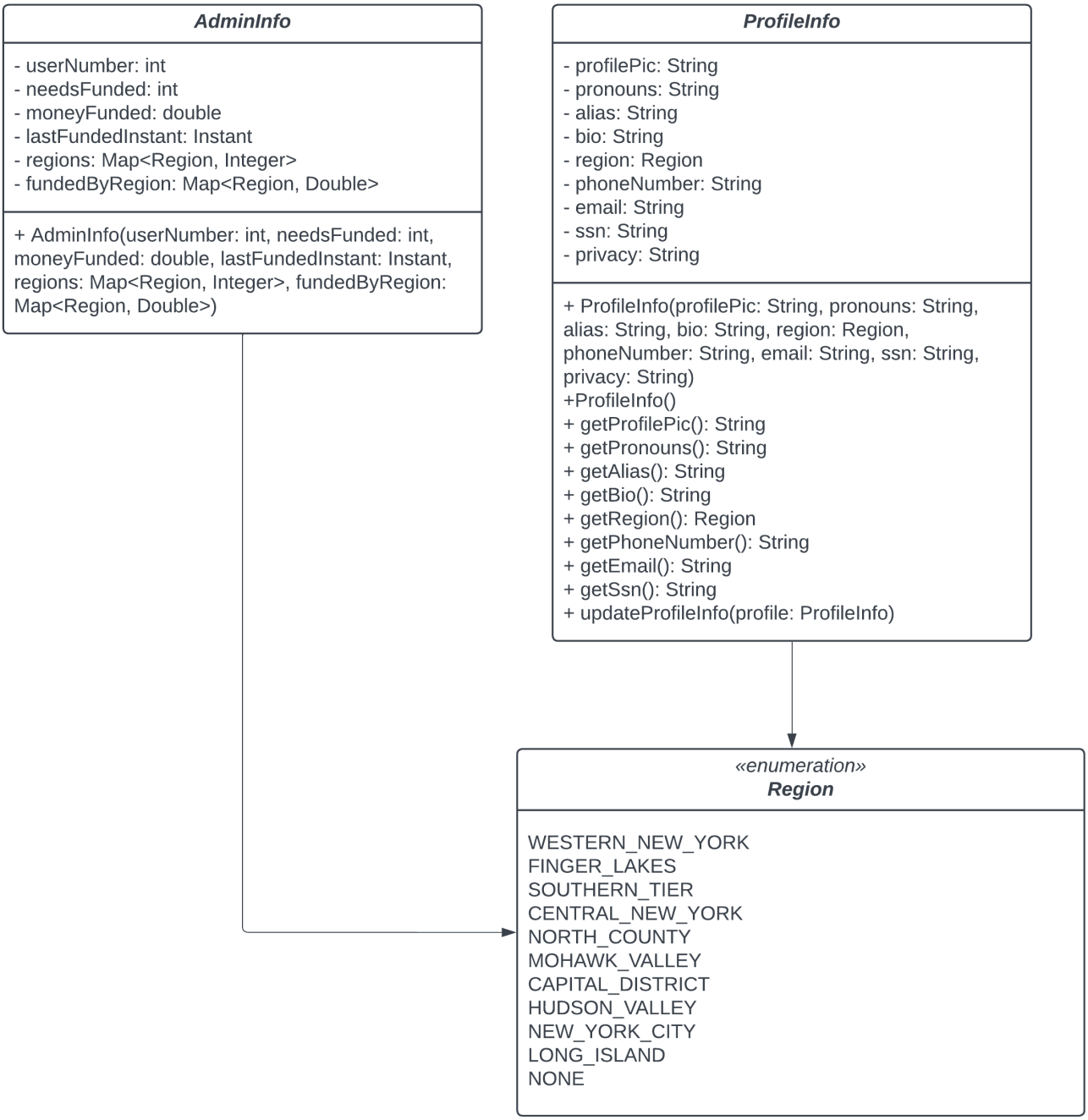
Full UML

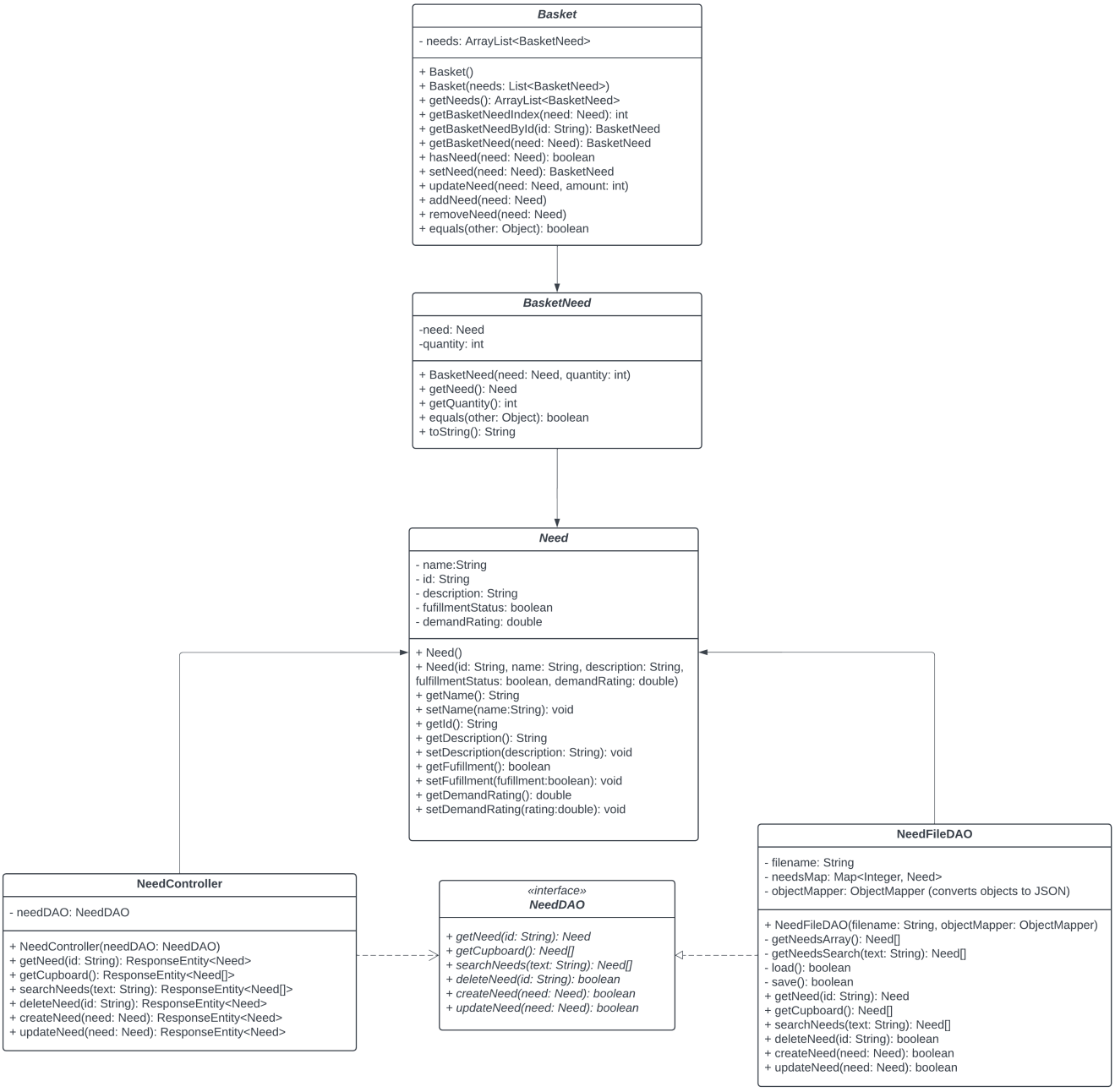


UML Breakdown


Account System
Info System
Need/Basket System


«interface»
AccountDAO
+ getNeeds(name: String): ArrayList<BasketNeed>
+ updateNeed(name: String, need: Need, amount: int): BasketNeed
+ checkout(name: String): boolean
+ createAccount(name: String): Account
+ getAccount(name: String): account
+ getGod(): account
+ addImage(mame: String, img: byte[]): String
+ getRankList(): List<Account>
+ getRankListTop3(): List<Account>
+ getProfileInfo(name: String): ProfileInfo
+ getRank(name: String): int
+ updateProfileInfo(name: String, profile: ProfileInfo): ProfileInfo
+ getUserStats(): AdminInfo
AccountController
- AccountDAO: NeedDAO
+ createAccount(name : String): ResponseEntity<Account>
+ updateNeed(name: String, amount: int, need: Need): ResponseEntity<BasketNeed>
+ checkout(name: String): ResponseEntity<BasketNeed>
+ getNeeds(name: String): ResponseEntity<ArrayList<BasketNeed>>
+ getProfileInfo(name: String): ResponseEntity<ProfileInfo>
+ getAccount(name: String): ResponseEntity<Account>
+ getAccountsSorted(): ReponseEntity<List<Account>>
+ getAccountsSortedTop3(): ReponseEntity<List<Account>>
+ getRank(name: String): ResponseEntity<Integer>
+ getGod(): ResponseEntity<Account>
+ handleImageUpload(name: String, imageFile: MultiPartFile): ResponseEntity<String>
+ updateProfileInfo(name: String, profile: ProfileInfo): ResponseEntity<ProfileInfo>
+getAdminInfo(): ResponseEntity<AdminInfo>
AccountFileDAO
- filename: String
- accountsMap: Map<String, Account>
- objectMapper: ObjectMapper (converts objects to JSON)
+ AccountFileDAO(filename: String, objectMapper: ObjectMapper, needDao: NeedDAO)
- getAccountsArray(containsText: String): Account[]
- getAccountsArray(): Account[]
- load(): boolean
- save(): boolean
+ createAccount(name: String): Account
+ getNeeds(name: String): ArrayList<BasketNeed>
+ updateNeed(name: String, need: Need, amount: int): BasketNeed
+ checkout(name: String): boolean
+ getAccount(name: String): account
+ getGod(): account
+ addImage(mame: String, img: byte[]): String
+ getRankList(): List<Account>
+ getRankListTop3(): List<Account>
+ getProfileInfo(name: String): ProfileInfo
+ getRank(name: String): int
+ updateProfileInfo(name: String, profile: ProfileInfo): ProfileInfo
+ getUserStats(): AdminInfo
Account
- name:String
- basket: Basket
- passwordHash: String
- moneyFunded: double
- needsFunded: int
- lastCheckoutInstant: Instant
- profileInfo: ProfileInfo
- isGod: boolean
+ Account(name: String, passwordHash: String)
+ Account(name: String), basket: Basket, profileInfo: ProfileInfo, passwordHash: String, imageLink: String)
+ Account(name: String), basket: Basket, profileInfo: ProfileInfo, passwordHash: String)
+ getName(): String
+ getBasket(): Basket
+ equals(other: Object): boolean
+ addMoneyFunded(amount: double): double
+ addNeedsFunded(quantity: int): int
+ setLastCheckoutInstant(instant: Instant)
+ setLastCheckoutInstant()
+ getLastCheckoutInstant(): Instant
+ getMoneyFunded(): double
+ getNeedsFunded(): int
+ getIsGod(): boolean
+ getPasswordHash(): String
+ setImageLink(imageLink: String)
+ getProfileInfo(): ProfileInfo
+ compareTo(other: Account): int
+ toString(): String


AdminInfo
- userNumber: int
- needsFunded: int
- moneyFunded: double
- lastFundedInstant: Instant
- regions: Map<Region, Integer>
- fundedByRegion: Map<Region, Double>
+ AdminInfo(userNumber: int, needsFunded: int, moneyFunded: double, lastFundedInstant: Instant, regions: Map<Region, Integer>, fundedByRegion: Map<Region, Double>)
ProfileInfo
- profilePic: String
- pronouns: String
- alias: String
- bio: String
- region: Region
- phoneNumber: String
- email: String
- ssn: String
- privacy: String
+ ProfileInfo(profilePic: String, pronouns: String, alias: String, bio: String, region: Region, phoneNumber: String, email: String, ssn: String, privacy: String)
+ProfileInfo()
+ getProfilePic(): String
+ getPronouns(): String
+ getAlias(): String
+ getBio(): String
+ getRegion(): Region
+ getPhoneNumber(): String
+ getEmail(): String
+ getSsn(): String
+ updateProfileInfo(profile: ProfileInfo)
«enumeration»
Region
WESTERN_NEW_YORK
FINGER_LAKES
SOUTHERN_TIER
CENTRAL_NEW_YORK
NORTH_COUNTY
MOHAWK_VALLEY
CAPITAL_DISTRICT
HUDSON_VALLEY
NEW_YORK_CITY
LONG_ISLAND
NONE



OO Design Principles

Championed Principles

Low Coupling: The principle of Low Coupling allows for modules that can interact without knowledge of each other's implementation.

Information Expert: Each class should be responsible for its own data.

The Single Responsibility

A class is considered to comply with the single responsibility principle if there is one and only one reason for the class to change. One “thing” is not necessarily well defined, but it refers to one group of related behaviors and states.

For example, each of our Java classes has a single function that it conforms to, and we've outlined those responsibilities below to showcase how each class remains independent to its purpose, and doesn't need to

change based on other components.

The only class that potentially violates the Single Resposibility is the AccountDAO, which requires changes when either the Account process *or* the basket-checkout proccess changes, as the checkout functionality requires both an Account basket and a Need cupboard.

NeedController: Only responsible for updating and fetching needs

NeedDAO: Only responsible for DAO processes related to Needs

NeedFileDAO: Same as NeedDAO but specialized

Need: Only needs to be updated if the structure of a need changes

AccountController: Only responsible for creating, updating, and getting Accounts.

AccountDAO: Only responsible for DAO processes related to Accounts (**and** checking out need baskets)

AccountFileDAO: Same as AccountDAO but specialized

Account: Only needs to be updated in the struture of account changes

Basket: Only needs to be updated in the structure of Basket changes

BasketNeed: Only needs to be updated if the structure of BasketNeed changes

Open/Closed

A class should be open to expansion but closed to modification. Expansion means that different components can be used in a class but the behavior of the class does not change. A good example of this is the NeedDAO, which does not need to be modified to add more behavior. Instead, we inherit NeedDAO and create a new object (NeedFileDao) which is an extension.

Our Need class does not comply with this principle. To add new behavior to a Need, we must change all the Needs within the Need database. For example, adding a "type" field to each Need would require us to migrate the entire database in order to preserve our data while supporting the new value of Need. We would also have to modify our methods and parsers for creating and testing Needs to support the new field.

Open/Closed is useful for maintaining backwards compatibility. By not modifying existing code, you minimize the chance of breaking old code. Instead you can build new independent functionality on top of the existing code. While the old code can run with what is now a limited feature set, the new code, which depends on new features, is also able to run - removing the need to refactor large parts of code (which depends on the thing being changed) when adding features.

Dependency Inversion/Injection

The principle of Dependency Inversion states that high level modules should not rely on low level modules. Rather, there should be levels of abstraction between these two tiers. This allows for looser coupling within the program. A good example of Dependency Inversion is the Model, View, View-Model Architecture. There are multiple instances where we implement this architecture throughout our program:

AccountController -> AccountDAO -> AccountFileDAO -> Account and similarly, NeedController -> NeedDAO -> NeedFileDAO -> Need

In both of these cases, the controller calls DAO methods, which is an abstraction of the concrete implementation FileDAO. The FileDAO calls model methods to alter instances of classes, and then saves those to our persistence system (A JSON file)

In both our controllers, the DAO abstractions are injected via the constructor. In many of our frontend modules, services and other modules are injected via the constructor as well.

Pure Fabrication

The principle of Pure Fabrication involves creating modules that are not within the problem domain in order to make implementation, cleaner, more reusable, and more efficient. Pure Fabrication is an effective to way to solve problems that violate the Single Responsibility principle. That is, if you find yourself writing a module that does more than one significant thing, you should probably make a different module to handle whatever it is you're implementing. That way if there is any refactoring later down the line you only have to touch the module that is relevant to your problem. Below I will list some parts of our application which adhere to the Pure Fabrication principle.

Our AccountFileDAO and NeedFileDAO are responsible for the storage and accessing of accounts and needs in our database, respectively. We did this so we could seperate runtime and persistence functionality- it would be messy to have one big class that held state, functions, and managed our storage system.

We also used the Pure Fabrication principle in many places on our Front End. We found it was useful to seperate modules up and use them throughout the program. For example, we has a level service that was solely responsible for determining a user's rank. We also had an authorization service, which other components used, to verify a user was allowed to access a specific page.

Static Code Analysis

While we're overall quite happy with our current production code, running SonarQube highlighted a few potential areas for improvement as we continue the development process. The most significant of these are outlined below.

1. Stacktrace printing in ProfileInfo.java. This line was added to be able to adequately handle the error that arises when an Illegal Access exception occurs, but printing the stack is not recommended for production code.

Shown in Basket.java:

src/.../com/ufund/api/ufundapi/model/ProfileInfo.java Open in IDE

```
121     public void updateProfileInfo(ProfileInfo info) {
122         if (info != null) {
123             for (Field field : ProfileInfo.class.getDeclaredFields()) {
124                 field.setAccessible(true); // Allow access to private fields
125                 try {
126                     Object value = field.get(info);
127                     if (value != null) {
128                         field.set(this, value);
129                     }
130                 } catch (IllegalAccessException e) {
131                     e.printStackTrace();
```

Make sure this debug feature is deactivated before delivering the code in production.

```
132                 }
133             }
134         }
135     }
136
137     /**
138      * {@inheritDoc}
139      */
140     @Override
141     public boolean equals(Object other){
```

2. Override hashCode methods. When we created our Model objects, we implemented equals() methods for comparison. However, it's recommended that all objects that would override an equals() also override hashCode() to support other types of comparisons, which may be relevant depending on potential model comparisons later in development.

Example in Basket.java:

ufund-api > src/.../com/ufund/api/ufundapi/model/Basket.java See all issues in this file

```
196 mbk986...
197     /**
198      * {@inheritDoc}
199      */
200     @Override
201     public boolean equals(Object other){
```

This class overrides "equals()" and should therefore also override "hashCode()".

3. Random number gen for message ID. We're using Typescript's Math.random(), which is potentially nonrandom and was highlighted as a potential security error, but should be secure enough for our current use case. These IDs are only used for message display, not handling any secure user data.

Shown in message.service.java:

src/app/message.service.ts Open in IDE

```
      }
    }
  }

  clear() {
    this.messages = [];
  }

  generateFakeUUID() {
    return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, function(c) {
      const r = Math.random() * 16 | 0, v = c == 'x' ? r : (r & 0x3 | 0x8);
```

Make sure that using this pseudorandom number generator is safe here.

```
      return v.toString(16);
    });
  }
}
```

4. Accessibility: There are a few changes in our HTML code that would make it more accessible, such as adding `<alt>` tags for images and KeyboardPress alternatives for our Click events. These changes would make the website more accessible for users who can't view images or who don't have access to a mouse or keypad.

Future Design Improvements

1. We should have used an authentication service from the start. This would have increased security, and made our job easier for setting username and password requirements, separating create account and login (which we did not do), and managing sessions.
2. We would like to have used a database rather than a file for storing data. We could have stored data in persistent storage rather than memory, greatly decreasing server resource usage. This would have allowed for easy sorting and searching of users, as well as efficiently finding calculated data for a user.
3. AccountDAO is dependent on NeedDAO for checkout. On checkout, AccountDAO has to modify the NeedDAO (deleting or decreasing need quantities). We believe there may be a more elegant solution, than directly calling the NeedDAO. Maybe we could pass a functional interface that is called to checkout a basket, which at runtime would call a NeedDAO but would be replaceable by any other functional interface. This interface would be returned by a factory method in NeedDAO (ie NeedDAO.getCashier()) and passed to the AccountDAO constructor. This decouples the two DAOs while maintaining the same functionality.
4. It would be better for usability to have a clear separation between logging in and signing up. We could then also tailor the error messages better, i.e. "username already taken" vs. "incorrect username or password".
5. There should be a confirmation dialog when removing a Need from the basket in case a user misclicks and then has to find the Need all over again or (worst case) gives up.
6. It would be easier for the Helper if there was a way to add larger or custom increments of a Need to their basket directly from the list of needs on the homepage.
7. It would be easier for the Helper if a summary of their basket was displayed on the homepage or from a dropdown button so that they can simultaneously see the list of Needs and what they are already planning to check out.

8. It may not be completely intuitive that setting one's alias changes their name on the leaderboard from "Anonymous" to the name they set, so a brief line of text explaining this would help with usability.

Testing

Acceptance Testing

When originally testing the acceptance criteria, we ran into a few problems with specific edge cases (For example, when an admin deletes a need, if that need is in a helpers basket, it would stay in a helper's basket, when the desired functionality was for it to be removed). So, we refined our implementation and tests and explored some more edge case tests (negative numbers of Needs, users checking out while the admin was changing Need values, confirmation popups, etc) to ensure the MVP was glitch-free.

Then, after retesting our acceptance criteria, we reached 100% acceptance critera completion within our acceptance critera spreadsheet, which is where we now stand as of the current implementation.

Unit Testing and Code Coverage

Our unit testing strategy was that whenever any backend functionality was implemented, before it was pushed to our development branch the author of that code would create a Jacoco report and ensure that our overall coverage hadn't dropped. If it had, then it was the author's responsibility to add any necessary tests before creating a PR of their code. Additionally, reviewers on a backend PR would ensure that all tests were passing before marking their approval.

Our code coverage target was **95%** or above for all tiers. We felt that this was high enough to ensure that the main functionality of our code was intact without slowing down our development by trying to figure out how to write tests for obscure branches. In the end, we ended up getting almost **100%** coverage accross the board, ensuring that all the methods in our backend are working as expected.

ufund-api

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.ufund.api.ufundapi		88%		n/a	1	4	2	7	1	4	0	2
com.ufund.api.ufundapi.model		99%		100%	0	132	2	261	0	77	0	7
com.ufund.api.ufundapi.persistence		100%		100%	0	74	0	197	0	34	0	2
com.ufund.api.ufundapi.controller		100%		100%	0	41	0	149	0	27	0	4
Total	8 of 2,643	99%	0 of 218	100%	1	251	4	614	1	142	0	15

Anomaly Note: The missing one percent in the model tier is a branch that catches an access error, given our current code structure it was evaluated to be a low-priority enough branch that no test was needed. The other missing coverage is the main method running the Spring application, which theoretically shouldn't ever fail.

Ongoing Rationale

> **[Sprint 1] decisions**
>
> - (2024/10/1) Switched from integer IDs or unique names for Needs to String UUIDs. This change was agreed upon to simplify the backend logic and limit conflicts based on hidden backend information.
> - (2024/10/1) Decided to keep our DAO as Need/NeedFile rather than Cupboard, as the Cupboard name is a concept for the customer and holds no signficance to working with the Need classes on the backend.
>
> **[Sprint 2] decisions**

- (2024/10/4) Need creation now fails with an error response if the numerical arguments (cost, quantity, demand) are less than 0. Given the realistic constraints of needs, those values should never be negative.
- (2024/10/11) Updated UI to use icons instead of text for managing needs buttons
- (2024/10/12) Renamed StorageService to AuthService for user authentication/login functionality
- (2024/10/16) Refactored BasketNeed to have a Need and a quantity instead of a UUID and a quantity
- (2024/10/20) Changed editing need in basket quantity to numerical input instead of increment/decrement methods
- (2024/10/20) Needs are not allowed to be added to the basket if they would overflow the maximum available quantity. We're not sure if the Paws and Claws foundation has the space to store surplus, and we don't want it to go to waste.
- (2024/10/20) All Needs are auto-sorted by Demand rating in the UI
- (2024/10/22) If two users both try to fund the same basket need at the same time and there's not enough to go around, the first one to fund the needs gets to fund them and the later person gets an error message.

[Sprint 3] decisions

- (2024/10/24) Refactored logout system. Previously, we were using local service variables to store authenticated usernames. However, on any page refresh/initialization these credentials were removed. The application now instead uses localStorage to store credentials so these persist across pages. This also allowed us to add an authentication guard to non-login pages, so indexing a route without authentication is cleaner.
- (2024/10/30) Decided to create a ProfileInfo model to store an accounts profile information since these values are only ever changed by the user and aesthetic
- (2024/10/31) Decided to use an md5 library to hash passwords so they are not stored in plaintext by our backend.
- (2024/11/2) Using the Cloudinary API for creating and storing image links in the cloud for profile images.
- (2024/11/5) Determine rank first by moneyFunded, then use the number of needsFunded, then consider whoever made the most recent donation to be a better rank
- (2024/11/5) Changed admin loggin functionality. Password is now required, and set to "adm1n!".
- (2024/11/6) User level-up no longer requires an account age factor because the focus should be more on the amount of money funded than the amount of time it took for a user to do it
- (2024/11/7) Leaderboard on home page only shows top three users, plus logged in user if not in top 3 users. If there is a u-fund god, they are displayed above the leaderboard.
- (2024/11/7) Decided to make a new component for viewing a profile that is not yours. There is a route that leads to this component, and localstorage which tracks which profile you attempt to visit
- (2024/11/8) Decided to do phone number and SNN formatting for profile info in angular rather than making the user do it so that they have an easier time updating their information
- (2024/11/9) Added more specific password validation messages on the login screen
- (2024/11/9) Added a new field to profileInfo, privacy, which determines whether other users can see your profile from the leaderboard component.
- (2024/11/10) When you tie the moneyFunded for the Top 5% Index of funders, you become a Master as well, even if cut off by to 5% math

> - (2024/11/10) When progressing towards the next level, your percentage is never rounded up to 100%
> - (2024/11/10) The admin can see user demographic statistics, but they should also be able to see the leaderboard. We gave them a tabular dashboard page so that all three of these menus are avilable.
> - (2024/11/11) The CSS on the Dashboard is difficult enough that it's not going to be presented in the mobile version of the app; admins on mobile will see text reminding them that they can check out the desktop web app to view the descriptive statistics.